

Roll No.

--	--	--	--	--	--	--	--	--	--

END SEMESTER EXAMINATION 2025

Name of the Course: B. Tech (CSE)
 Name of the Paper: Compiler Design
 Time: 3 Hours

Semester: VI
 Paper Code: TCS 601
 Maximum Marks: 100

Note: -

- (i) All questions are compulsory.
 (ii) Answer any two sub-questions among a, b & c in each main question.
 (iii) Total marks in each main question are twenty.
 (iv) Each question carries 10 marks.

Q1 (20 marks)	
(a) Write short notes on any two: i. Usage of symbol table w.r.t each phase of compiler ii. Compare and contrast single and double buffering scheme. iii. Input buffering vs cross compiler	CO1
(b) (a) Differentiate between lexeme, pattern, and token with suitable examples. (b) Analyze the following C program and count the total number of tokens it contains. Provide a detailed explanation of the tokens identified. <pre>#include <stdio.h> int main() { int y = 10.0; y = 10 + y; printf("Value of y = %d\n", y); return 0; }</pre>	
(c) Write a regular expression for the following patterns and explain their use in lexical analysis: • Identifiers in C. • Integer and floating-point constants. • String literals enclosed in double quotes.	
Q2 (20 marks)	
(a) Consider the following grammar: 1. $E \rightarrow E - E / E + E$ 2. $E \rightarrow E / E / E * E$ 3. $E \rightarrow (E)$ 4. $E \rightarrow \text{num}$ Parse the input string 'num+num/num' using the operator precedence parsing algorithm. Clearly show all the steps used in the function table.	CO2
(b) Analyze the time-space complexity of LL(1) parser. Check whether the following grammar is LL(1) or not by making parsing table. $S \rightarrow aX$ $X \rightarrow Ad \mid BX$	

	A $\rightarrow aY$ Y $\rightarrow \epsilon \mid b$ B $\rightarrow ccd \mid ddc$															
	(c) Prove that the given grammar is not SLR but CLR. S $\rightarrow L = R$ S $\rightarrow R$ R $\rightarrow L$ L $\rightarrow *RL$ L $\rightarrow id$															
Q3	(20 marks)															
(a)	i. Differentiate between Syntax-Directed Definitions (SDD) and Syntax-Directed Translation (SDT) with suitable examples. ii. Construct an SDT for a desk calculator that evaluates expressions with the following expression. '7*(4+3)-6/2'	CO3														
(b)	Given the expression: $a=(x+y)*(z-w)/(x+y)*(z-w)$ (a) Write the three-address code (TAC) for the given expression in the following representations: - Triple - Quadruple - Indirect Triple															
(c)	Given the following grammar for simple arithmetic expressions: <table border="1"> <thead> <tr> <th>PRODUCTION</th> <th>SEMANTIC RULES</th> </tr> </thead> <tbody> <tr> <td>1) $E \rightarrow E_1 + T$</td> <td>$E.\text{node} = \text{new Node}('+', E_1.\text{node}, T.\text{node})$</td> </tr> <tr> <td>2) $E \rightarrow E_1 - T$</td> <td>$E.\text{node} = \text{new Node}('-', E_1.\text{node}, T.\text{node})$</td> </tr> <tr> <td>3) $E \rightarrow T$</td> <td>$E.\text{node} = T.\text{node}$</td> </tr> <tr> <td>4) $T \rightarrow (E)$</td> <td>$T.\text{node} = E.\text{node}$</td> </tr> <tr> <td>5) $T \rightarrow id$</td> <td>$T.\text{node} = \text{new Leaf}(id, id.entry)$</td> </tr> <tr> <td>6) $T \rightarrow \text{num}$</td> <td>$T.\text{node} = \text{new Leaf}(\text{num}, \text{num.val})$</td> </tr> </tbody> </table> Construct a syntax tree for the input expression 5+3*2 and evaluate it using the attribute rules for S-attributes. Show all the steps of evaluation and how the attributes are propagated.	PRODUCTION	SEMANTIC RULES	1) $E \rightarrow E_1 + T$	$E.\text{node} = \text{new Node}('+', E_1.\text{node}, T.\text{node})$	2) $E \rightarrow E_1 - T$	$E.\text{node} = \text{new Node}('-', E_1.\text{node}, T.\text{node})$	3) $E \rightarrow T$	$E.\text{node} = T.\text{node}$	4) $T \rightarrow (E)$	$T.\text{node} = E.\text{node}$	5) $T \rightarrow id$	$T.\text{node} = \text{new Leaf}(id, id.entry)$	6) $T \rightarrow \text{num}$	$T.\text{node} = \text{new Leaf}(\text{num}, \text{num.val})$	
PRODUCTION	SEMANTIC RULES															
1) $E \rightarrow E_1 + T$	$E.\text{node} = \text{new Node}('+', E_1.\text{node}, T.\text{node})$															
2) $E \rightarrow E_1 - T$	$E.\text{node} = \text{new Node}('-', E_1.\text{node}, T.\text{node})$															
3) $E \rightarrow T$	$E.\text{node} = T.\text{node}$															
4) $T \rightarrow (E)$	$T.\text{node} = E.\text{node}$															
5) $T \rightarrow id$	$T.\text{node} = \text{new Leaf}(id, id.entry)$															
6) $T \rightarrow \text{num}$	$T.\text{node} = \text{new Leaf}(\text{num}, \text{num.val})$															
Q4	(20 marks)															
(a)	Consider the following 3-address code for a program: 1) t1 = x + y 2) t2 = t1 * z 3) t3 = w - t2 4) t4 = t3 * t3 5) t5 = x * t4 6) result = t5 + t2 7) p = result * 2 8) if p > 100 goto (10)	CO4														

9) $q = p - 50$ 10) $result = q + p$ Construct the Directed Acyclic Graph (DAG) for the above 3-address code. Clearly indicate the shared expressions and eliminate redundant computations in the DAG.	(b) Consider the following 3-address code for a given program: <pre> 1) a = 10 2) b = 20 3) c = a + b 4) if c > 15 goto (8) 5) d = b - 5 6) e = d * 2 7) f = e + 3 8) g = c / 2 9) h = f + g 10) if h > 25 goto (12) 11) i = h + 10 12) j = i - 5 13) k = j * 3 14) goto (13) 15) calling the program (a) Identify and mark the Leader Statements for this program. (b) Determine the Basic Blocks in the given program. (c) Construct the Control Flow Graph (CFG) for the given program, illustrating the flow of control between basic blocks.</pre> (c) Give detailed classification of various code optimization techniques with suitable example.	Q5 (20 marks) (a) Write a short note on any two of the following with an example: - Peephole Optimization - Loop optimization Techniques - Machine dependent code optimization (b) Consider the following intermediate code for a simple program: <pre> 1) t1 = a + b 2) t2 = c * d 3) t3 = t1 - t2 4) if t3 > 0 goto (8) 5) e = t1 + t2 6) f = e - t3 7) if f == 0 goto (10) 8) g = f + t3 9) end 10) result = g * 2</pre>
---	---	---

(a) Generate the target code (in assembly language) for the given intermediate code using a register-based model. Assume the following: - a, b, c, d, e, f, g are variables. - Registers R0, R1, R2, R3, R4 are available for intermediate storage. (b) Discuss how the target code generation process optimizes the usage of registers and reduces memory accesses.	(c) Consider the following program. <pre> fact(x) { int f = 1; for (i = 3; i <= x; i++) f = f * i; return f; }</pre> Compute three address codes for the same and find the Basic Blocks with the help of leader statements.
---	--